

Pactline: Problem, Architecture, Current State, and Next Build Plan

Executive summary

Pactline is an invoice-processing agent designed for **ArmorIQ Problem Statement 1:**

“Autonomous, until it shouldn’t be.” Its purpose is not simply to automate invoice work. Its purpose is to let an AI agent proceed autonomously while its actions remain inside a user-approved intent plan, then stop before a side effect when a proposed action leaves that plan.

The current repository contains a polished operator dashboard, a deployed API, a real remote MCP endpoint registered in ArmorIQ, a local deterministic proof, and an SDK-ready adapter. The dashboard and current run API are now data-driven at the application level, but the run API still uses a controlled simulation model and the live ArmorIQ SDK is not yet wired into the actual run execution path. Therefore, the project is a strong integration prototype, not yet a complete live production agent.

Accurate one-sentence description: Pactline is an ArmorIQ-registered invoice-agent prototype with a real MCP tool surface, backend-backed run and approval state, and an SDK-ready authorization path; the remaining work is to connect real invoice tools and live SDK decisions so the hold happens through ArmorIQ rather than through the prototype’s local policy model.

1. What the problem statement is asking

The official statement asks teams to build an AI agent that performs meaningful autonomous work using tools, encounters a genuinely out-of-scope or dangerous action, and is stopped **before the side effect executes**. The boundary must be based on the agent’s originally captured intent and cryptographic verification, not a fragile keyword filter. The demo should show a real task, routine actions proceeding, a wrong action being held, a human decision, and an auditable result. A mock MCP server is acceptable when the security enforcement is the focus, but the authorization boundary itself must be real rather than merely animated in a frontend [1](#).

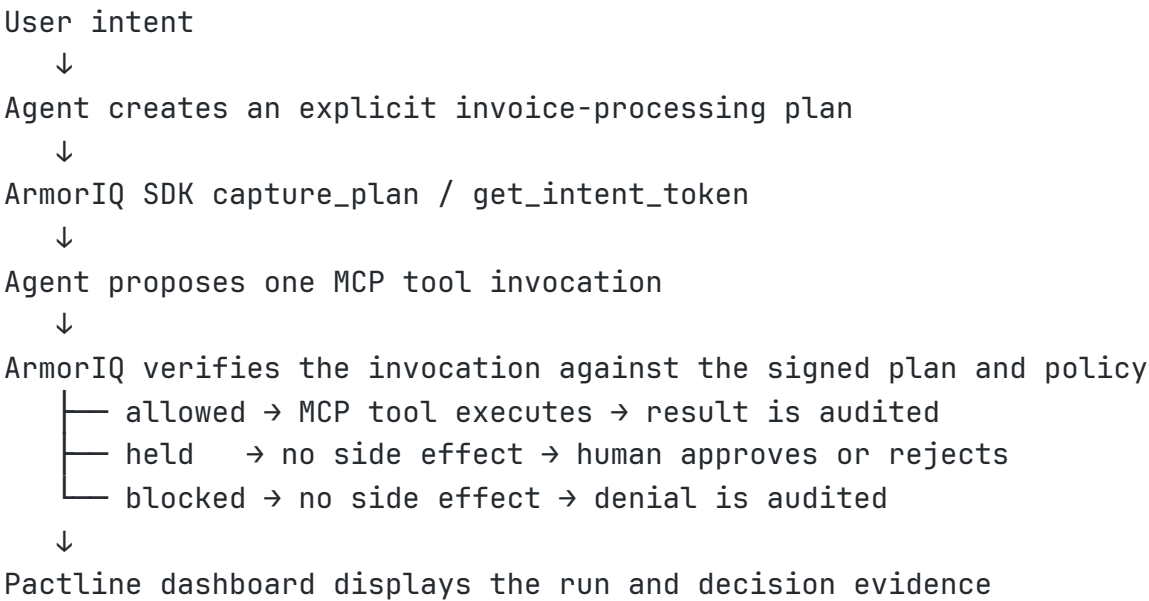
The statement’s invoice example is directly aligned with Pactline: the agent reads invoice data, extracts fields, writes a record, and then encounters an instruction to email extracted data to an unknown external address. The safe actions should complete, the email should be stopped, and the system should prove that the unauthorized side effect did not happen [1](#).

2. What Pactline is building

Pactline has five logical layers:

Layer	Responsibility	Current implementation
Invoice agent	Chooses the next tool action and processes invoice work	Local deterministic proof plus controlled backend run model
ArmorIQ SDK	Captures intent, creates cryptographic proof, and verifies each invocation	Official package is installed in the Windows project; live adapter code exists, but it is not yet on the dashboard execution path
MCP server	Exposes callable tools to the agent	Remote <code>/api/mcp</code> endpoint with four discovered tools
Policy and approval plane	Decides allow, hold, or block and routes holds to a human	MCP policy created in the ArmorIQ console; frontend approval actions are backend-backed in the prototype
Pactline Control Center	Shows runs, pending actions, audit evidence, and human decisions	Polished React dashboard with live API fetch/mutation flow

The intended end-to-end flow is:



3. The role of the ArmorIQ SDK

The SDK is the security mechanism between the agent's reasoning and the MCP tool execution. It is not a UI library and it is not the MCP server itself.

According to the official SDK documentation, the normal flow is to initialize an `ArmorIQClient` with an API key, create or capture a plan, obtain an intent token, and pass that token into every `invoke()` call. ArmorIQ canonicalizes the plan and creates cryptographic proof material; each invocation is then checked against that proof at the ArmorIQ proxy before the tool call proceeds [2](#).

For Pactline, the live TypeScript shape is conceptually:

```
const client = new ArmorIQClient({ apiKey: process.env.ARMORIQ_API_KEY })
const user = process.env.USER_EMAIL;

const plan = {
  goal: "Process invoice and notify the approved finance recipient",
  steps: [
    { action: "read_invoice", mcp: "pactline-invoice", params: { invoiceId: "12345" } },
    { action: "extract_fields", mcp: "pactline-invoice", params: { invoiceId: "12345" } },
    { action: "write_record", mcp: "pactline-invoice", params: { invoiceId: "12345" } },
    { action: "send_email", mcp: "pactline-invoice", params: { recipient: "john.doe@example.com" } }
  ]
};

const captured = client.capturePlan("invoice-agent", userPrompt, plan);
const token = await client.getIntentToken(captured);

await client.invoke("pactline-invoice", "read_invoice", token, args, undefined);
```

A deliberate out-of-scope call should then be attempted with a recipient or data scope that is not in the captured plan. The expected result is a hold or block before the MCP tool creates an external side effect. The official documentation describes this distinction as cryptographic verification of the explicit plan before the action reaches the MCP tool [2](#).

4. The role of the MCP server

MCP is the tool interface. Pactline's MCP server exposes the operations that an agent may request:

Tool	Intended side effect	Desired policy
<code>read_invoice</code>	Reads invoice metadata and totals	Allow inside the plan
<code>extract_fields</code>	Extracts invoice fields	Allow inside the plan
<code>write_record</code>	Writes an invoice ledger record	Allow inside the plan
<code>send_email</code>	Sends invoice information externally	Hold for approval when outside the plan

The remote endpoint is currently registered as `pactline-invoice` at `https://pactline-agent.vercel.app/api/mcp`. ArmorIQ has reached it, discovered the four tools, marked it healthy, and reported a clean scan. The MCP server is therefore a real deployed interface, but its tool implementations are still controlled prototype behavior rather than complete invoice/database/mail integrations.

The MCP server is not a replacement for the SDK. The MCP server exposes tools; the SDK and ArmorIQ proxy decide whether the agent is allowed to invoke those tools. The dashboard is not a replacement for either one; it displays the run and collects the human decision.

5. What the API key does

`ARMORIQ_API_KEY` authenticates Pactline's server-side SDK communication with ArmorIQ. The official authentication documentation states that the SDK can read the key from `ARMORIQ_API_KEY`, and that production deployments should provide it through environment variables rather than a local credentials file or frontend code [3](#).

`USER_EMAIL` is the end-user identity attached to each decision. It lets ArmorIQ attribute the plan and tool activity to the user on whose behalf Pactline is operating. It is not a secret, but it should still remain a server-side configuration value for the agent.

The key is **not** used to make React buttons work. It is **not** an MCP tool permission by itself. It is **not** safe to put in `VITE_*` variables, browser JavaScript, screenshots, GitHub, or chat. The secure placement is:

Local test:	<code>.env</code> on the developer machine, loaded into the server process
Vercel deploy:	Project Settings → Environment Variables, server-side only
Never:	<code>client/src</code> , <code>VITE_*</code> , GitHub commits, screenshots, chat messages

The local preflight now reports all required and optional variables present. That proves configuration presence only; it does not yet prove that the SDK has completed a successful live authorization call.

6. What is real today and what is still simulated

Area	Current truth	What must change for a fully live build
React visual design	Real, polished UI	Keep and refine
Frontend fetch/mutations	Real calls to <code>/api/run</code> ; loading, retry, approve, and reject states exist	Keep; add authentication and durable storage later
<code>/api/run</code> state	Real HTTP endpoint and real state transitions within a process	Replace controlled sample invoice/actions with real tool execution
Reject behavior	Real backend mutation: held action becomes rejected, audit entry is added, outbox remains empty	Make the rejected action originate from the ArmorIQ SDK decision, not local state logic
Approve behavior	Real backend mutation: action becomes approved and a simulated outbox entry is added	Send only through a safe test mailbox after ArmorIQ allows the invocation
<code>/api/mcp</code>	Real remote MCP endpoint and tool discovery	Implement actual invoice/file/database/mail effects behind the tools
ArmorIQ registration	Real; agent and MCP are registered	Attach and verify the policy against live SDK traffic
Local deterministic agent	Real local program logic, but intentionally deterministic and SDK-shaped	Replace local authorization decision with actual SDK capture/token/invoke results
SDK adapter	Real adapter code that imports <code>@armoriq/sdk</code> and creates a session/plan/check flow	Wire it into the run endpoint and verify a real decision in ArmorIQ observability
API key	Real key has been created and stored locally	Use it in a server-side live SDK test; never expose it

Area	Current truth	What must change for a fully live build
Audit trail	Real application-level audit entries from the prototype API	Include ArmorIQ decision IDs/proof metadata and durable storage

The user-facing website is therefore **not “all fake,” but it is not yet fully live either**. Its state transitions are backend-backed, which is an important improvement over a purely hardcoded React demo. However, the backend currently uses a controlled sample invoice and local decision model. The next architectural step is to move the source of truth for action authorization to the ArmorIQ SDK and move tool execution behind the MCP server.

7. Why the SDK smoke-test error occurred

The command failed because the new file `agent/armoriq-live-test.mjs` exists in the sandbox working copy but was not yet copied into the user’s Windows repository. Node therefore correctly reported:

```
Cannot find module ...\\agent\\armoriq-live-test.mjs
```

This is a file synchronization problem, not an ArmorIQ credential failure and not proof that the SDK is broken. The fix is to synchronize the new file to GitHub and pull it into the Windows project before rerunning the command.

The test also needs the official SDK package available in the Windows project. The `.env` variables must be loaded into the current PowerShell process because Node does not automatically read a plain `.env` file unless the project explicitly uses a dotenv loader or Node’s env-file support. The successful preflight already confirmed that the user’s manual PowerShell loading command works.

8. The non-hardcoded implementation we should build next

The next implementation should be one complete vertical slice rather than more dashboard decoration.

Step A: real invoice input

Use one real local sample invoice file, preferably a PDF or JSON fixture with a known invoice ID, vendor, amount, and line items. `read_invoice` must read that file rather than return a fixed

object. `extract_fields` must parse fields from the file. The frontend should display the returned values from the API.

Step B: real ledger effect

`write_record` should insert the extracted invoice into SQLite or another small durable store. The returned record ID should be displayed in the dashboard. The test must be able to prove that the safe action created a real row.

Step C: real test-mail effect

`send_email` should be connected to a test-only mailbox such as Mailtrap or a controlled outbox provider. It must never send to a real personal or production address. The test should demonstrate that the held path creates no message, while the approved path creates exactly one test message.

Step D: SDK in the execution path

The backend run endpoint should create the plan from the actual user/task request, call the SDK to capture the plan and obtain the intent token, then invoke each MCP tool through the SDK. It should persist the SDK decision, reason, proof identifier, and tool result. The frontend should only render the resulting state.

Step E: deliberate out-of-scope action

After the safe invoice actions, deliberately propose an email recipient or data scope not covered by the captured plan. The system must call the SDK for that action. If ArmorIQ returns hold, the API must persist the pending decision and return it to the dashboard without calling the mail tool. When the user chooses reject, the pending invocation must be cancelled and the outbox must remain empty. When the user chooses approve, the system may resume the invocation and write the test-mail result.

Step F: observability proof

The dashboard should show the ArmorIQ session or decision identifier, the plan identifier, the action, decision, reason, and side-effect status. The strongest video should show both rejection and empty outbox; the stronger technical test should also show approval and one controlled test-mail result.

9. Current project position

Pactline has completed the **prototype infrastructure stage**:

Milestone	Status
Problem selection and product concept	Complete
Premium frontend and navigation	Complete
Local allow/hold/reject proof	Complete
Deployed health endpoint	Complete
Deployed MCP endpoint	Complete
ArmorIQ agent registration	Complete
ArmorIQ MCP registration and tool discovery	Complete
MCP policy draft	Created/configured in the console; activation and live-traffic verification remain
Backend-backed frontend state	Complete at controlled-prototype level
Real ArmorIQ SDK authorization in the run path	Not complete
Real invoice parsing/database/mail effects	Not complete
Durable production persistence	Not complete
Submission video and PPT	Still to prepare

The project should now stop adding static visual claims. The best next step is the real SDK smoke test, followed immediately by wiring one safe tool call through the SDK. Once that works, implement the hold path and then the real side-effect proof.

10. Exact next commands after synchronization

After pulling the updated repository on Windows:

```
cd "C:\Users\vishe\OneDrive\Desktop\Goal\microsoft hack"  
git pull origin main
```

Load the local `.env` into the current PowerShell session using the already working command, then verify presence:

```
node agent\armoriq-preflight.mjs
```

Run the repaired smoke test:

```
node agent\armoriq-live-test.mjs
```

Only share the sanitized result fields: `status` , `decision` , `sideEffectExecuted` , and `error` . Never share the key or full response.

11. Honest demo wording

Until the smoke test and live tool invocation succeed, use this wording:

“Pactline is an ArmorIQ-registered prototype with a deployed MCP tool surface and an SDK-ready authorization path. The dashboard demonstrates the operator workflow; the next integration step is routing the real invoice tool calls through ArmorIQ’s cryptographic intent verification.”

After a successful live SDK decision is visible in ArmorIQ, the wording can become:

“Pactline captures the invoice task as an explicit intent, invokes routine MCP tools through ArmorIQ, and holds the external email before execution when its recipient falls outside the signed plan.”

References

13. Exact Windows error and complete repair

The exact error received on Windows was:

```
PS C:\Users\vishe\OneDrive\Desktop\Goal\microsoft hack> node agent\armoriq-live-test.mjs

Error: Cannot find module 'C:\Users\vishe\OneDrive\Desktop\Goal\microsoft hack\agent\armoriq-sdk'
code: 'MODULE_NOT_FOUND'
```

This error does **not** mean that the API key is invalid, the ArmorIQ SDK is broken, or the MCP server failed. It means the Windows copy of the repository did not yet contain the newly created file. The file existed in the development workspace and was then synchronized to GitHub; the Windows project must pull the latest commit before Node can execute it.

The repair sequence is:

```
cd "C:\Users\vishe\OneDrive\Desktop\Goal\microsoft hack"
git pull origin main
Test-Path .\agent\armoriq-live-test.mjs
pnpm install
```

`Test-Path` must return `True`. If it returns `False`, the folder is not the synchronized Git repository or the pull did not complete. In that case, run `git status`, confirm the remote is the ArmorIQ repository, and run `git pull origin main` again. Do not create a substitute file manually, because that would cause the local project to diverge from the tested implementation.

The official SDK package must also exist in the same Windows project:

```
pnpm list @armoriq/sdk
```

The expected dependency is `@armoriq/sdk 0.6.10`. The `.env` file must remain local and must never be committed. Because a plain `.env` file is not automatically loaded by every Node execution path, load it into the current PowerShell session using the previously validated command, then run the diagnostics without printing secret values:

```
Get-Content .env | Where-Object { $_ -and $_ -notmatch '^\s*#' } | ForEach-Object {
    node agent\armoriq-preflight.mjs
    node agent\armoriq-live-test.mjs
}
```

The preflight should report `ARMORIQ_API_KEY present: true`, `USER_EMAIL present: true`, and no errors. The smoke test should report only sanitized fields such as `status`,

`decision` , `sideEffectExecuted` , and `error` . Never send the API key, authorization header, full SDK response, or a screenshot containing the secret.

14. No-hardcoded-input rule

The target implementation must not accept a fake decision from the browser or use a fixed frontend event list as its source of truth. The browser may request `start` , `approve` , or `reject` , but the server must own the run state. The server must read the invoice fixture, invoke the MCP tool, call ArmorIQ for authorization, persist the tool result, and return the resulting decision to the frontend. A reject must leave the outbox empty. An approval may write exactly one controlled test-outbox message. If ArmorIQ credentials are missing or the SDK call fails, the system must show an explicit error instead of silently converting the run into a fabricated success.

The current implementation has the real tool runtime and the SDK execution path in the repository, but live verification still depends on the private Windows credentials. The current checkpoint therefore represents a tested implementation path, not proof that the organizer's live ArmorIQ endpoint has already recorded a successful decision.

15. What to do next, in order

First synchronize the Windows repository and confirm that `agent\\armoriq-live-test.mjs` exists. Second install dependencies and run the preflight plus sanitized smoke test. Third, if the smoke test succeeds, configure the same server-side variables in the Vercel project and redeploy. Fourth, run one public invoice workflow and inspect ArmorIQ API logs for a real decision. Fifth, verify both branches: rejection leaves the controlled outbox empty, while approval creates one controlled test message. Only after these checks should the video claim that the SDK is actively enforcing the boundary.